

Parallel and Distributed Computing: Homework 5

Objectives: The main goal of this exercise is to experimentally measure the performance of the parallelization of a data transfer process using different versions of fine grain and coarse grain multi-threading.

Consider the problem of in-memory transposition of a given square matrix. In a single processor, the pseudo-code to in-memory transpose a matrix A is shown in algorithm 1.

Algorithm 1: Matrix Transposition

```
Input: A;           // Input Matrix
1 Function (A):
2   for  $0 \leq i \leq n-1$  do
3     for  $i+1 \leq j \leq n-1$  do
4        $temp \leftarrow A_{ij};$ 
5        $A_{ij} \leftarrow A_{ji};$ 
6        $A_{ji} \leftarrow temp;$ 
7     end
8   end
9 return
```

Note that this code scans the matrix by rows and exchanges symmetric elements with respect to the main diagonal focusing on individual elements of the upper triangular section of the matrix A , excluding the main diagonal.

One way of parallelizing the transposition in a multi-core environment is to share A among different threads, so they all access it, and do part of the job concurrently. Elements of A would be read by the threads following a coordinated indexing scheme to guarantee each element of A is accessed once and only once by only one thread.

Levels of coarseness: Threads may have different levels of coarseness. On the one hand, they could be light or **fine grain**, doing little work each. For example, we could create a number of threads, and assign only one exchange to each of them; each thread repetitively requesting more work until the transposition is finished.

On the other hand, threads can be made heavy, or **coarse grain**, by giving a big

amount of work to each thread. For example, we could create a number of threads and assign a several rows of the matrix to each; an extreme example would be having just one thread doing all the work.

Several questions arise with this parameter: Is there a “*best*” coarseness, and under what conditions? What are the benefits/trade-offs towards each extreme?

Assignment

Consider a matrix A of size n , meaning that it contains $n \times n$ elements stored in row major form, where $2 \leq n \leq 10,000$. Using the template code provided, implement the sequential and parallel transposer functions.

mat_sqtrans_seq(mat)

This single-threaded transposer function shall perform an in-place transposition, as shown in Alg. 1. It receives one parameter: the matrix to transpose **mat**.

mat_sqtrans_par(mat, grain, threads)

This multi-threaded transposer function also shall perform an in-place transposition, receives three parameters:

- **mat:** Data structure representing the matrix to transpose. It is shared across all the threads performing the transposition.
- **grain:** Coarseness level. The number of exchanges that each thread performs before requesting more work.
- **threads:** Number of threads to use in the transposition.

Debugging: You may change any file in the template, but you will only submit

`transpose.c` with the implementation of `mat_sqtrans_seq` and `mat_sqtrans_par`, perhaps with more helper functions, all contained in the same `transpose.c` file. **Make sure your code works when the only modified file is `transpose.c`.**

Makefile: The provided template has a Makefile with three targets:

- **build:** compile and link the binary from the source code.
- **test:** test that the sequential and parallel versions are the same.
- **clean:** delete build/test artifacts.

So to build and test your results, you can simply run “`make build test`”. Note that the test only check that the sequential and parallel implementations output the same. That does not ensure correctness.

Achieving parallelism: Feel free to use `pthread` or `openMP` to implement the parallel version of the function. To obtain meaningful experimental results, a computer with a multi-core processor should be used. A four-core processor is strongly recommended.

Report

Write a report discussing each of these 5 points in one page each. *Do not forget to include your name and student ID number on the top of the first page of your report.*

1. A characterization of the single threaded transposer execution time as the size of the matrix increases. Add plots and figures as necessary to explain your results.
2. A characterization of the multi threaded transposer execution time for a **fine-grain** configuration. Keep the number of exchanges per thread constant, `grain=1`. Keep the size of the matrix constant, `n=7,000`. Vary the number of threads in `{1, 2, 3, 4, 5, 6, 8, 16, 24, 32, 64, 128}`. Add

plots and figures as necessary to explain your results.

3. A characterization of the multi threaded transposer execution time for a **coarse-grain** configuration. Experiment with different values for the parameter `grain`, particularly `grain=n` and `grain=(n×n)/threads`. Keep the size of the matrix constant, `n=7,000`. Vary the number of threads in `{1, 2, 3, 4, 5, 6, 8, 16, 24, 32, 64, 128}`. Add plots and figures as necessary to explain your results.
4. Fixing `n=7,000`, graph the speedup obtained by using the fine (`grain=1`) and coarse (`grain=n`) grain multi-thread version with respect to the sequential, as the number of threads in the multi-threaded varies in `{1, 2, 3, 4, 6, 8, 16, 32, 64, 128}`. Explain your results.
5. Consider sizes $10 \leq n \leq 10000$. Graph the speedup obtained by the fastest coarse-grain transposer with respect to the sequential version. Explain your results.

Submission

Submit **one** zip file named `hw5.zip`, containing **exactly 3 files**. Make sure the zip file does **not** include sub-directories (as Mac’s default compression tool does) or extra files:

1. **transpose.c:** The edited C source template with your well commented implementation of the single and multi threaded transposer. **Do not submit any other source code file.**
2. **report.pdf:** The digitally produced report.
3. **team.txt:** Text file with two lines containing the UCINetID and name of each student in your team. Each line will have this format:
`<UCINetID>: <firstName> <lastName>`